

RAG Implementation Guide

How the React + Express PDF chat app works end to end

Project: RAGReactApp | Scope: upload, parse, chunk, embed, retrieve, prompt, answer

1. What RAG Means

RAG stands for Retrieval-Augmented Generation. The idea is simple: instead of asking the LLM to answer only from its built-in knowledge, we first retrieve relevant text from the user's uploaded document, then pass that text into the model as context.

In this project, the uploaded PDF is parsed into text, split into chunks, embedded into vectors, stored in a vector store, retrieved again when the user asks a question, and finally combined into a prompt that Gemini uses to generate the answer.

2. What This App Does

- Frontend: React + Vite chat interface with PDF upload, chat history, markdown rendering, loading state, and error handling.
- Backend: Express API for upload and chat, plus services for parsing, chunking, embeddings, vector storage, and RAG orchestration.
- AI: Gemini is used twice, once for embeddings and once for final answer generation.
- Storage: Pinecone is supported when configured, and a local in-memory fallback is used when Pinecone credentials are absent.

3. End-to-End Flow

- 1 User uploads a PDF from the frontend.
- 2 The backend saves the file and parses it into text.
- 3 The extracted text is chunked into smaller pieces with overlap.
- 4 Each chunk is converted into a Gemini embedding.
- 5 The embeddings are stored in Pinecone or local memory.
- 6 When the user asks a question, the question is also embedded.
- 7 The backend retrieves the most relevant chunks from vector storage.
- 8 Those chunks are assembled into a context block.
- 9 The context plus the question is sent to Gemini 2.5 Flash.
- 10 Gemini produces the final answer, which is returned to the UI.

4. File-by-File Implementation

Backend entry points

- Server/src/server.js starts the Express app and listens on the configured port.
- Server/src/app.js wires middleware, CORS, JSON parsing, and the /upload and /chat routes.
- Server/src/routes/uploadRoutes.js handles PDF uploads with multer.
- Server/src/routes/chatRoutes.js handles user questions.

Parsing and upload

- Server/src/middleware/uploadMiddleware.js configures multer disk storage and PDF-only validation.
- Server/src/controllers/uploadController.js receives the file, parses it, stores the latest document in memory, chunks it, and indexes it.
- Server/src/services/pdfService.js reads the PDF and extracts text using pdf-parse.

Chunking

- Server/src/utils/chunkText.js splits document text into section-aware chunks.
- It detects common headings like Experience, Projects, Education, Skills, Summary, and keeps those sections grouped.
- Long sections are split again with overlap so context is preserved across chunk boundaries.

Embeddings and vectors

- Server/src/services/embeddingService.js calls Gemini embedding model gemini-embedding-001.
- Server/src/services/vectorService.js stores vectors in Pinecone when configured.
- If Pinecone is not configured, it falls back to an in-memory vector store for local testing.

RAG orchestration

- Server/src/services/ragService.js creates embeddings for each chunk during upload.
- It retrieves top matches for a user question and sorts them into a readable order.
- For broad document questions, it can use the full uploaded document context instead of only top-k chunks.
- It then prompts Gemini 2.5 Flash with the context and the question.

Frontend

- client/src/pages/Home.jsx shows the document upload button and chat window.
- client/src/hooks/useChat.js manages chat state, upload state, errors, and document info.
- client/src/api/chatApi.js sends multipart upload requests and chat requests to the backend.

- `client/src/components/ChatMessage.jsx` renders markdown responses.
- `client/src/components/ChatWindow.jsx` auto-scrolls to the latest message.

5. How Chunking Works

Chunking is important because long PDFs are too large to send directly to the embedding model or the chat model as one block. The chunker first normalizes the text, then looks for likely section headings. If it sees a heading, it starts a new section. That keeps important structure together, especially in resumes, reports, manuals, and letters.

If a section is still too large, it is split into overlapping chunks. Overlap matters because key facts often sit near a boundary. Without overlap, one company name or one sentence can fall between chunks and be harder to retrieve later.

6. How Embeddings Work

For each chunk, the backend sends the text to Gemini embedding model `gemini-embedding-001`. The model converts text into a numeric vector that represents semantic meaning. Similar ideas produce nearby vectors, which is what makes similarity search possible.

The code for this is in `Server/src/services/embeddingService.js`. It calls `ai.models.embedContent` with contents: `[text]`. The response returns embedding values, which are stored along with the chunk metadata.

7. How Retrieval Works

When the user asks a question, the backend embeds the question too. That question vector is compared against the stored document vectors. The vector store returns the top matches by semantic similarity.

In local mode, the app computes cosine similarity in memory. In Pinecone mode, Pinecone handles the similarity search. In both cases, the goal is the same: find the document chunks that best match the question.

8. How Context Is Built

The best matching chunks are not used individually. The app orders them by page and chunk index so the context reads naturally. It can also include neighboring chunks around the match to preserve surrounding meaning.

For broad questions, the app may use the full uploaded text or the combined chunk set. This reduces omissions for questions like 'summarize this document' or 'list all key details.'

9. How the LLM Gets the Answer

The final prompt includes instructions plus the retrieved context and the user question. Gemini 2.5 Flash then generates the response. The model is told not to invent details and to prefer direct extraction from the context.

The answer is returned to the frontend and rendered in markdown. That is why formatted bullets, headings, and structure can appear nicely in the chat window.

10. Important Functions Used

Function	File	Role
extractTextFromPdf	pdfService.js	Reads the PDF file and extracts plain text.
chunkText	chunkText.js	Splits PDF text into sections and overlapping chunks.
generateEmbedding	embeddingService.js	Converts text into vector embeddings using Gemini.
upsertVectors / queryVectors	vectorService.js	Stores and retrieves vectors from Pinecone or memory.
createEmbeddingsForDocument	ragService.js	Indexes every chunk from the uploaded PDF.
answerWithRag	ragService.js	Embeds the question, retrieves context, and calls the LLM.
uploadPDF	uploadController.js	Coordinates upload, parsing, storing, and indexing.

11. Why This Pipeline Is Reliable

- It preserves document structure better than raw chunk splitting.
- It stores both content and metadata, which improves retrieval order and traceability.
- It supports local mode without Pinecone, so development stays easy.
- It can switch to full-document context for broad questions, which improves completeness.

12. Current Limitations

- The local fallback is per server process and resets when the backend restarts.
- Very noisy OCR-style PDFs can still produce imperfect extracted text.
- If you want persistent production search, Pinecone should be configured.

13. Summary

The app implements a practical RAG loop: upload, parse, chunk, embed, retrieve, assemble context, prompt Gemini, and return the response. The code is organized into small reusable services, so each part of the pipeline can be improved independently.